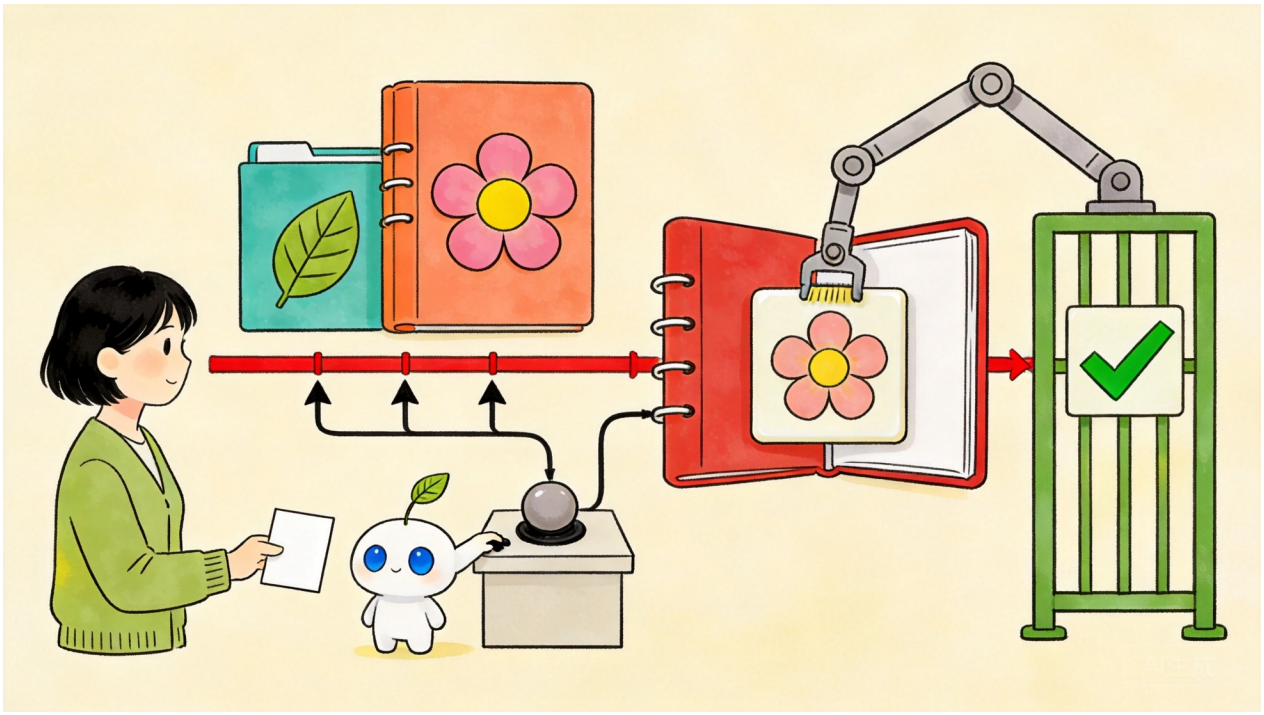
提示词不只是“怎么问”：Agent 如何靠一套指令完成复杂工作？

“通过 ASKS 智能知识系统看 AI Agent”系列第 5 篇

ASKS（Agent-Driven Scientific Knowledge System）是我们的智能知识系统。

项目代码：github.com/ranshiju/Ran-ASKS

本文通过与 ASKS 系统交互完成。



同一句任务，只有配上一整套工作规程，才可能稳定地变成行动

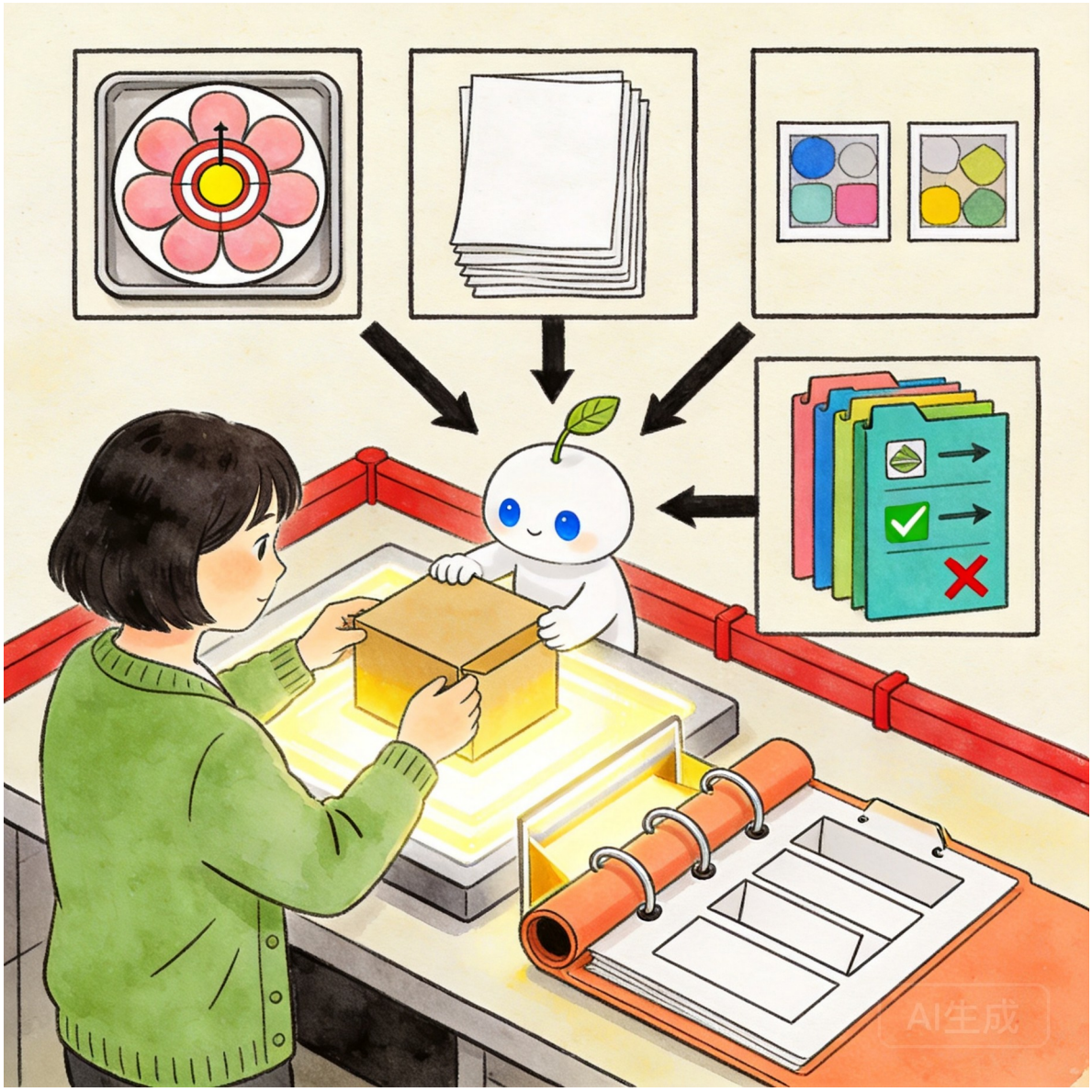
假设你对两个使用同一种大模型的系统说：

“把这篇论文整理进知识库。”

第一个系统写出一段流畅摘要，然后停下来。第二个系统保存原文、查看已有知识、整理新内容、检查来源，最后告诉你改了什么。

差别未必来自模型，也未必来自一句更“聪明”的问法。第二个系统背后，很可能还有一套你没有逐字说出的工作规程：什么材料可以动，先做哪一步，结果写成什么结构，何时调用工具，遇到风险何时停下来。

这些交给模型的指令和上下文，都属于广义的提示词。到了 Agent 时代，理解提示词的关键，也从“怎么问得巧”变成了“怎样把工作交代清楚”。



提示词把目标、材料、示例、边界和交付格式组织成模型本次工作的上下文

提示词究竟是什么：模型这一轮看到了什么？

最简单的提示词，是我们输入聊天框的一句话。但在一次模型调用中，提示词可以理解为交给模型看的全部指令和相关信息，往往远不止这一句。

仍以“把这篇论文整理进知识库”为例。用户只说了十几个字，ASKS 交给模型的工作包还可能包括：

- 这一次具体要完成什么；

- 从论文中选出的相关内容，以及知识库里已有的相关页面；
- 项目的基本规则，例如原始材料不能随意改动，重要结论必须能找到来源；
- 可以使用哪些工具，以及结果应当采用什么结构；
- 当前做到哪一步，完成后需要接受什么检查。

它就像交给一位临时加入项目的聪明同事一个文件夹：里面既有任务单，也有材料、规则和交付模板。提示词不是“咒语”，而是模型完成这一轮工作时拿到的信息包。

工具本身并不是提示词。告诉模型“有哪些工具、什么时候可以使用”属于提示词；工具是否真的有权限读写文件，则由模型之外的程序决定。

提示词工程是什么：怎样让同类任务稳定完成？

知道提示词里可以放什么，还只是第一步。提示词工程关心的是：怎样设计、测试和改进这个工作包，让模型面对一批相似任务时，不靠偶然发挥，也能比较稳定地按照要求工作。

ASKS 收入论文时，用户可能只说：“把这篇论文整理进知识库。”如果直接把这句话交给模型，它也许能写出一段流畅摘要，但结果可能每次不同：这次保留了研究方法，下次却漏掉；有时标明结论来自哪里，有时只留下一个无法核对的概括。

因此，ASKS 会先准备当前任务所需的材料、输出格式和工作规则，再让 Agent 处理需要理解的部分。假如 Agent 写好了摘要，普通程序却发现某条重要结论没有标清来源，这次改动就不会直接进入知识库。系统会把问题明确反馈给 Agent：“这条结论缺少来源，请回到论文中找到依据，定向修正。”修正后的结果还要再次接受同样的检查。

这就是提示词工程：它不是寻找一句更神奇的问法，而是把任务说明、输入材料、输出结构、检查标准和失败反馈组织成可以重复使用、不断改进的工作办法。到了 Agent 时代，提示词不仅要说明“怎样回答”，还要说明“怎样工作”，以及什么时候应该停下来接受检查。

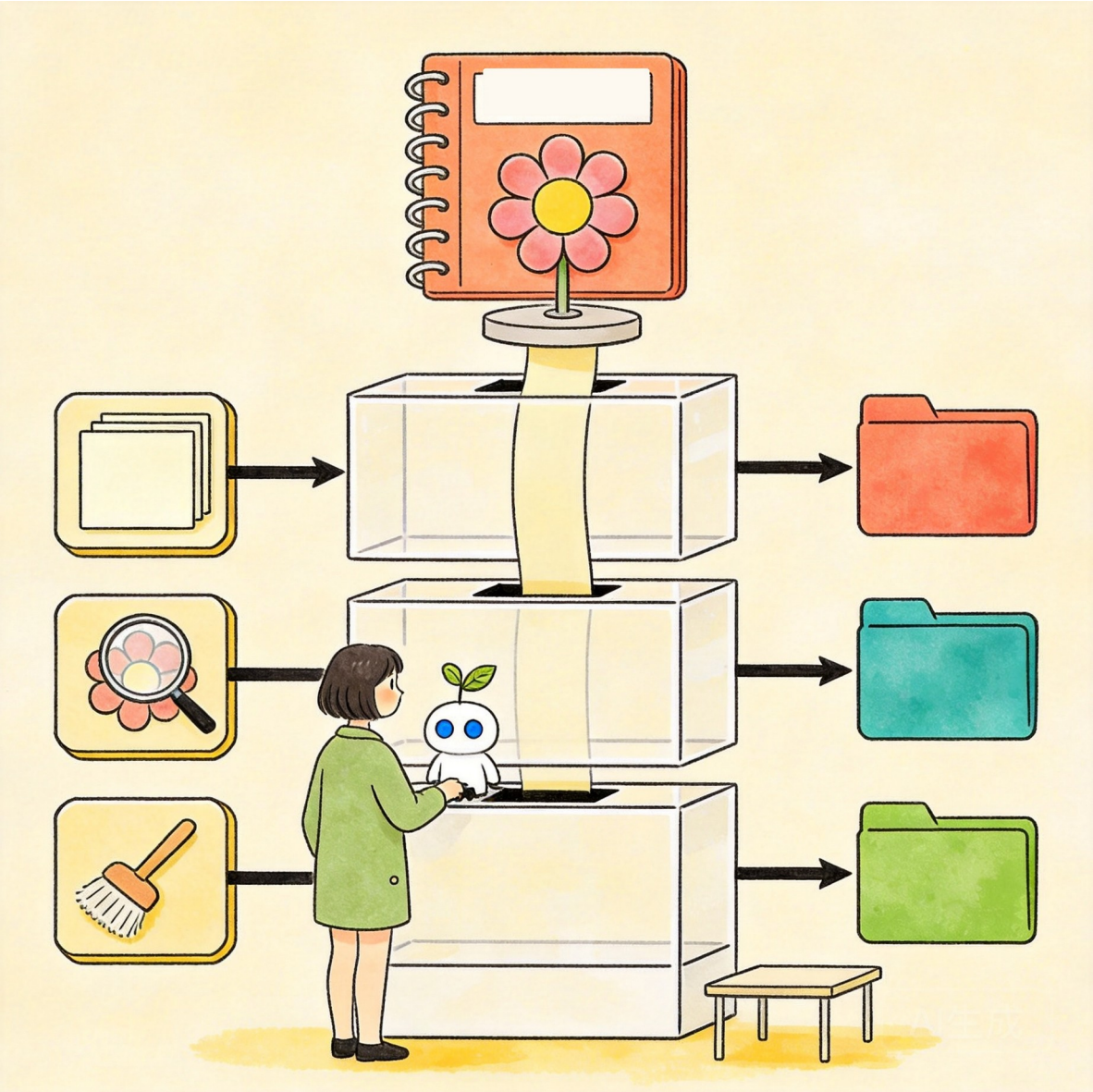
Karpathy 的 Wiki：把规则写成长期说明书

Andrej Karpathy 在公开的 LLM Wiki 方法说明中，提出由大模型持续维护个人 Wiki 的做法。他特别强调一份可以长期演化的 `schema` 文档，例如 `CLAUDE.md` 或 `AGENTS.md`。

这里的 schema 不是数据库表格，而是一份给模型看的工作说明书。它告诉模型：Wiki 怎样组织，页面遵循什么约定，收入新材料、查询问题和日常检查分别应该怎么做。

可以把它想成图书馆的工作手册。新管理员第一次来，不必靠猜测理解每个书架；手册把共同规则留下来。人和 Agent 在实践中发现新问题，还可以继续修改这份手册。于是，经验不只留在某次聊天里，而成为下一次任务也能使用的规则。

这一步很重要：提示词开始从一次性的问话，变成系统可以保存、版本化和持续改进的操作知识。



长期说明书保存共同规则，派发器再为不同任务取出相关章节

提示词派发：不要每次搬来整本手册

说明书越来越厚以后，另一个问题出现了：每次都把全部规则交给模型，既浪费上下文，也容易让无关要求互相干扰。

更实用的办法是先判断任务属于哪一类，再取出相关的那一章。这就是提示词派发。

如果用户要收录一篇论文，系统加载“收入新材料”的任务卡；如果用户在提问，就加载“查询与证据”的任务卡；如果用户要检查知识库，则加载“日常体检”的任务卡。共同的底线一直保留，专用规则按任务出现。

这有点像医院分诊：所有人遵守同一套基本制度，但验血、拍片和手术拿到的流程单并不相同。所谓派发，并不神秘，它只是让正确的任务在正确的时候拿到正确的说明。

ASKS 怎样使用提示词？

前面的论文入库例子，展示了一次任务怎样从说明走向检查。把视角拉远，会发现这些说明并不都塞在同一段提示词里，而是分布在几个层次。

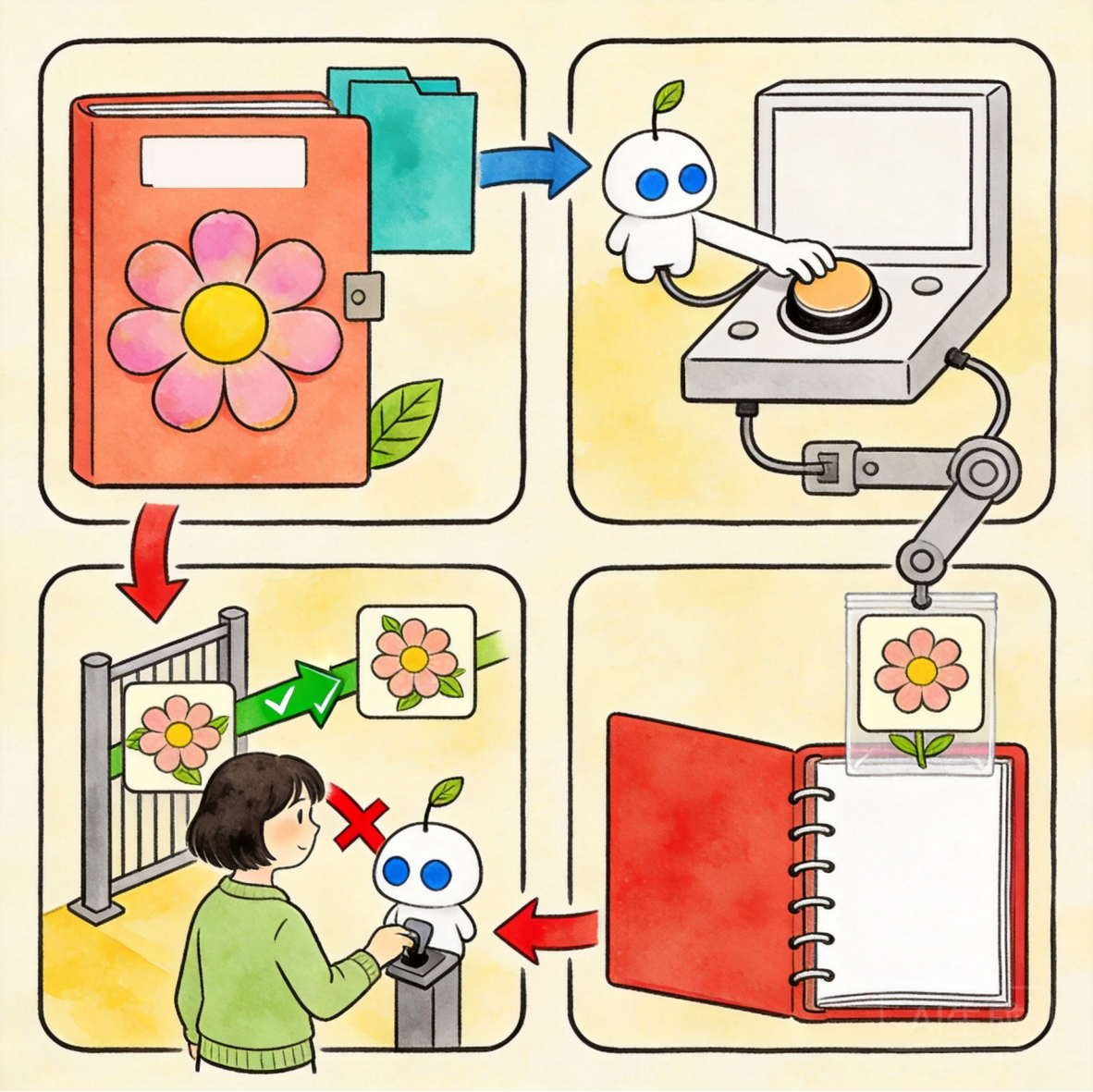
在我们的智能知识系统 ASKS 中，用户的简短要求不会被直接塞给模型，然后期待模型一次猜对所有事情。系统把说明分成几层：

首先，总规则规定不能随意改动原始材料，知识结论必须能找到来源。这相当于项目的基本章程。

接着，派发器判断这是“收入新材料”的任务，只加载这一任务所需的操作卡；如果任务进行到某一步还需要模型处理，Agent 收到的工作包只保留当前输入、预期输出、必须遵守的格式和完成后的下一步。

然后，模型负责需要理解的部分，例如判断论文讲了什么、哪些信息值得进入 Wiki；工具负责读取和写入；普通程序则检查来源是否齐全、结构是否合规、是否出现可以机械识别的重复。检查不通过，整次改动可以被挡住，Agent 再根据反馈修正。

这里没有一个“万能提示词”。更像是一套分层规程：长期章程保证方向一致，任务卡减少无关信息，输出结构让结果便于检查，工具把判断变成动作。



在 ASKS 中，提示词指导 Agent，工具执行动作，校验器把守可机械判断的规则

提示词能指导，却不能代替制度

提示词写着“不要修改原始材料”，不等于文件在技术上就一定改不了；提示词要求“不得编造”，也不等于模型从此不会理解错。

真正可靠的系统，还需要把关键边界做进模型之外：用权限限制能动什么，用受管工具规定怎样操作，用结构校验发现明确违规，用事务在失败时撤回改动，并在重要语义判断上保留人的确认。

因此，提示词、程序和人承担的是不同工作：

- 提示词把目标、知识和做事方法交代给 Agent；
- 程序执行确定动作，并守住能够明确编码的边界；
- 人处理价值取舍、重要风险和机器难以判断的含义。

这也解释了 Agent 时代提示词为什么如此重要，又为什么不能被神化。

> 提示词不再只是一句“怎么问”，而是模型参与复杂工作的操作规程；它要能被保存、按任务派发，并与工具和校验共同组成系统。

当提示词从一句灵感变成可维护的规则，人与 AI 的关系也随之改变：我们不只是在向一个模型索取答案，而是在把自己的工作方法逐步教给一个可以行动的系统。

ASKS arXiv 论文

- Shi-Ju Ran, Kun Zhang, Xi Wu, Liu-Si Yang, and Wen-Jun Li. LLMs Interpret, Embeddings Organize, Graphs Emerge: Agent-Driven Compilation of Scientific Knowledge. arXiv preprint, arXiv:2608.29612 (2026). ASKS 的系统论文，介绍来源保留、持续知识编译和可重放更新。

相关论文推荐

- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. ACM Computing Surveys, Vol. 55, No. 9, Article 195, pp. 1-35 (2023). 系统梳理提示方法从任务说明、示例到自动优化的发展，适合进一步理解提示词工程的基本版图。
- Laria Reynolds and Kyle McDonell. Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm. Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems, Article 314, pp. 1-7 (2021). 较早把提示设计讨论为一种面向大模型的编程方式，帮助理解提示词为何会从一句问话发展成工作规程。